



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 5
по курсу «Языки и методы программирования»
«Монады в языке Java»

Студент группы ИУ9-22Б Ивенкова О. В.

Преподаватель Посевин Д. П.

Москва 2024

1 Задание

Во время выполнения лабораторной работы требуется разработать на языке Java один из классов, перечисленных в таблице, которая приведена ниже. В каждом классе нужно реализовать по крайней мере два метода: первый метод должен возвращать `Stream`, а второй – `Optional`. Операции, выполняемые каждым методом, указаны в вариантах задания. В методе `main` вспомогательного класса `Test` нужно продемонстрировать работоспособность разработанного класса, осуществив группировку содержимого потока, возвращаемого первым методом, с помощью группирующего коллектора. В исходном коде (включая класс `Test`) запрещено использовать циклы и рекурсию.

1. Множество неравенств вида $x > a$, где x – имя переменной, а a – целое число, с операциями:

1. порождение на основе множества неравенств потока имён переменных, которые могут принимать только положительные значения;
2. вычисление минимального значения указанной переменной, удовлетворяющего всем неравенствам, в которые она входит.

Проверить работу первой операции нужно путём

2. Множество чисел, представленных объектами класса `BigInteger`, с операциями:

1. порождение потока, составленного из тех чисел множества, которые точно не являются простыми (см. метод `isProbablePrime`), отсортированного по возрастанию;
2. поиск числа, который делится на все остальные числа множества без остатка;
3. добавление числа.

Проверить работу второй операции нужно путём ранжирования чисел по длине в битах.

2 Результаты

Исходный код программы представлен в листингах 1–7.

Листинг 1 — Реализация вспомогательного класса Neravenstvo первой задачи

```
1 class Neravenstvo {
2     String name;
3     int number;
4     Neravenstvo (String name, int number) {
5         this.name = name;
6         this.number = number;
7     }
8 }
```

Листинг 2 — Реализация класса, заданного в первой задаче

```
1 import java.util.*;
2 import java.util.stream.Stream;
3 class NeravTable {
4     HashMap<String, Neravenstvo> Table;
5     NeravTable () {
6         Table = new HashMap<>();
7     }
8     void add(String name, int number) {
9         if (Table.containsKey(name)) {
10             if (number > Table.get(name).number) {
11                 Table.put(name, new Neravenstvo(name, number));
12             }
13         } else {
14             Table.put(name, new Neravenstvo(name, number));
15         }
16     }
17     public Stream<String> nameStream() {
18         ArrayList<String> result = new ArrayList<>();
19         Table
20             .entrySet()
21             .stream()
22             .filter(x -> x.getValue().number > 0)
23             .forEach(x -> result.add(x.getValue().name));
24         return result.stream();
25     }
26     public Optional<Integer> getNerav(String v) {
27         Optional<Integer> result = Optional.empty();
28         Optional<Map.Entry<String, Neravenstvo>> tmp = Table
29             .entrySet()
30             .stream()
31             .filter(x -> x.getValue().name.equals(v))
32             .findFirst();
33         if (tmp.isPresent()) {
34             result = Optional.of(tmp.get().getValue().number);
35         }
36         return result;
37     }
38 }
```

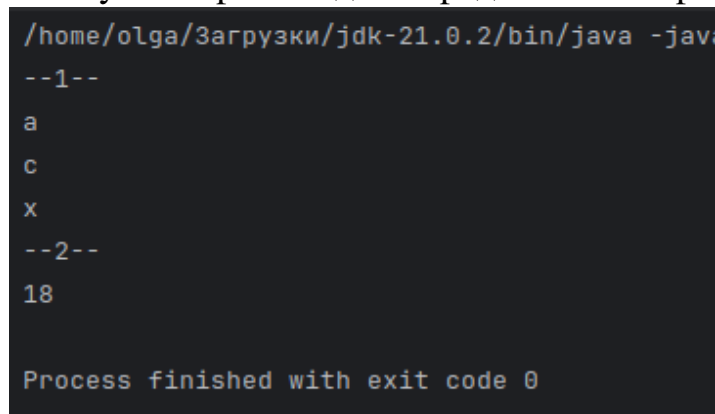
Листинг 3 — Реализация группирующего коллектора первой задачи

```
1 class NeravComparator implements Comparator<String> {  
2     public int compare(String a, String b) {  
3         char a0, b0;  
4         a0 = a.charAt(0);  
5         b0 = b.charAt(0);  
6         if (a0 > b0) { return 1; }  
7         if (a0 == b0) { return 0; }  
8         return -1;  
9     }  
10 }
```

Листинг 4 — Реализация класса Test первой задачи

```
1 public class Test {  
2     public static void main(String[] args) {  
3         NeravTable t = new NeravTable();  
4         t.add("a", -10);  
5         t.add("x", 6);  
6         t.add("x", -7);  
7         t.add("c", 2);  
8         t.add("a", 18);  
9         t.add("d", -6);  
10        t.add("d", -10);  
11        t.add("a", -6);  
12        t.add("x", 0);  
13        System.out.println("--1--");  
14        t.nameStream().sorted(new NeravComparator()).forEach(System.out  
:: println);  
15        System.out.println("--2--");  
16        System.out.println(t.getNerav("a").get());  
17    }  
18 }
```

Результат запуска первой задачи представлен на рисунке 1.



```
/home/olga/Загрузки/jdk-21.0.2/bin/java -java  
--1--  
a  
c  
x  
--2--  
18  
  
Process finished with exit code 0
```

Рис. 1 — Результат

Листинг 5 — Реализация класса, заданного во второй задаче

```

1 import java.util.*;
2 import java.util.stream.Stream;
3 import java.math.BigInteger;
4 class BigInterSet {
5     private List<BigInteger> numbers;
6     public BigInterSet() {
7         numbers = new ArrayList<>();
8     }
9     public void addNumber(BigInteger number) {
10         numbers.add(number);
11     }
12     public Stream<BigInteger> nonPrimeNumbersStream() {
13         ArrayList<BigInteger> result = new ArrayList<>();
14         numbers
15             .stream()
16             .filter(x -> !x.isProbablePrime(100))
17             .sorted()
18             .forEach(x -> result.add(x));
19         return result.stream();
20     }
21     public Optional<BigInteger> findDivisibleNumber() {
22         Optional<BigInteger> result = Optional.empty();
23         Optional<BigInteger> tmp = numbers
24             .stream()
25             .filter(num -> numbers
26                 .stream()
27                 .allMatch(divisor -> divisor.equals(num) || num.
28                     remainder(divisor).equals(BigInteger.ZERO)))
29             .findFirst();
30         if (tmp.isPresent()) {
31             result = Optional.of(tmp.get());
32         }
33         return result;
34     }
35 }

```

Листинг 6 — Реализация группирующего коллектора второй задачи

```

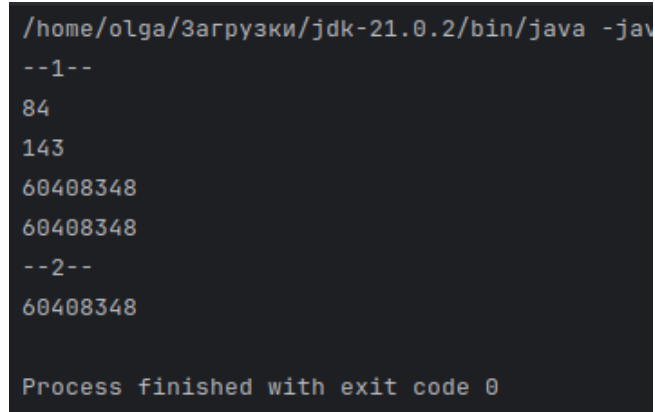
1 class BigIntComparator implements Comparator<BigInteger> {
2     public int compare(BigInteger a, BigInteger b) {
3         int bitsA = a.bitCount();
4         int bitsB = b.bitCount();
5         if (bitsA > bitsB) {
6             return 1;
7         } else if (bitsA < bitsB) {
8             return -1;
9         } else {
10             return 0;
11         }
12     }
13 }

```

Листинг 7 — Реализация класса Test второй задачи

```
1 public class Test {
2     public static void main(String[] args) {
3
4         BigIntegerSet s = new BigIntegerSet();
5         s.addNumber(BigInteger.valueOf(13));
6         s.addNumber(BigInteger.valueOf(11));
7         s.addNumber(BigInteger.valueOf(143)); // 11 * 13
8         s.addNumber(BigInteger.valueOf(60408348));
9         s.addNumber(BigInteger.valueOf(84));
10        s.addNumber(BigInteger.valueOf(47));
11        s.addNumber(BigInteger.valueOf(107));
12        s.addNumber(BigInteger.valueOf(11));
13        s.addNumber(BigInteger.valueOf(60408348));
14        System.out.println("--1--");
15        s.nonPrimeNumbersStream().sorted(new BigIntComparator()).forEach
16        (System.out::println);
17        System.out.println("--2--");
18        System.out.println(s.findDivisibleNumber().get());
19    }
```

Результат запуска второй задачи представлен на рисунке 2.



```
/home/olga/Загрузки/jdk-21.0.2/bin/java -jav
--1--
84
143
60408348
60408348
--2--
60408348

Process finished with exit code 0
```

Рис. 2 — Результат